



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

جمع/تفریق کننده ممیز شناور

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴-۱۴۰۳

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

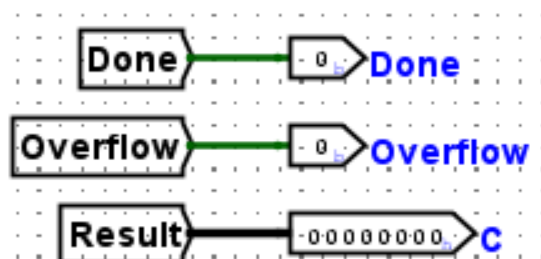
۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	مراحل طراحی و ساختار مدار	۲
۱.۳	Unpacker	۳
۲.۳	Exp_Aligner	۳
۳.۳	Big_ALU	۵
۴.۳	Normalizer	۶
۵.۳	واحد CU	۸
۶.۳	Main	۹
۴	نکات و جزئیات	۱۰
۵	بررسی Test Case ها	۱۰
۱.۵	تست اول: جمع ساده	۱۰
۲.۵	تست دوم: تفریق ساده و نرمال سازی	۱۰
۳.۵	تست سوم: اختلاف نمای زیاد	۱۰
۴.۵	تست چهارم: تولید عدد منفی	۱۱
۵.۵	تست پنجم: نرمال سازی عمیق	۱۱
۶.۵	تست ششم: جمع دو عدد منفی	۱۱
۷.۵	تست هفتم: اختلاف نمای بسیار زیاد	۱۲
۸.۵	تست هشتم: اورفلو	۱۲
۶	نتیجه گیری	۱۲

۱ مقدمه و هدف آزمایش

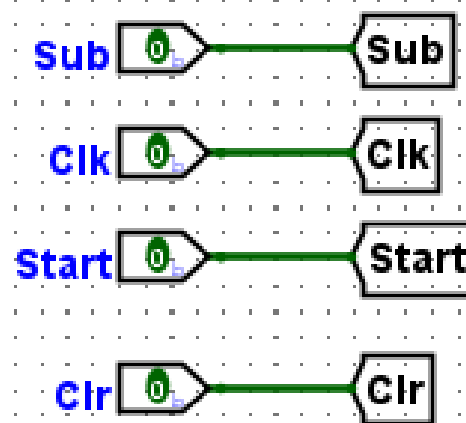
در این آزمایش، مدار ۳۲ بیتی جمع و تفریق‌کننده ممیز شناور را مبتنی بر استاندارد IEEE-754 طراحی و شبیه‌سازی می‌کنیم. هدف اصلی ما پیاده‌سازی یک معماری ترتیبی است که قادر باشد با دریافت سیگنال Start، دو عملوند A و B را بر اساس وضعیت پین sub با یکدیگر جمع یا تفریق کند.

۲ شرح آزمایش

روند کلی کار در این پیاده‌سازی به گونه‌ای است که ابتدا برای انجام محاسبات صحیح، فرآیند هم‌ترازی بر روی نمای دو عدد صورت می‌گیرد و سپس عملیات اصلی بر روی بخش کسری اعمال می‌شود. این طراحی پس از طی کردن مراحل محاسباتی، نتیجه نهایی را به صورت نرمالایز روی پین خروجی C قرار می‌دهد. همچنین برای مدیریت وضعیت سیستم در طول اجرای این مدار، اتمام موفقیت‌آمیز عملیات را با فعال شدن سیگنال Done به سایر بخش‌ها اعلام می‌کنیم و در صورتی که خطای اورفلو رخ دهد، این خطا را با یک کردن پین Overflow نمایش می‌دهیم.



شکل ۲: سیگنال‌های خروجی



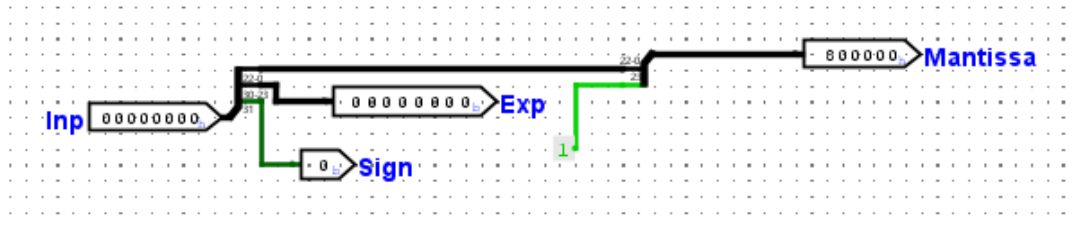
شکل ۱: سیگنال‌های ورودی

۳ مراحل طراحی و ساختار مدار

برای مدیریت پیچیدگی، معماری این مدار را به پنج زیرمدار مجزا تقسیم کردیم. در ادامه ساختار و وظیفه هر زیرمدار را بررسی می‌کنیم:

۱.۳ Unpacker

در این بلوک، ورودی‌های ۳۲ بیتی را دریافت کرده و آن‌ها را به سه بخش استاندارد تفکیک می‌کنیم: یک بیت Sign، ۸ بیت Exponent و ۲۳ بیت Mantissa. همچنین برای اینکه عملوند کامل ۲۴ بیتی داشته باشیم، با استفاده از یک اسپیلیتر، یک بیت با ارزش یک به عنوان بیت با ارزش به Mantissa اضافه می‌کنیم. در نهایت هر سه بخش تولید شده را به عنوان خروجی به پین‌های خروجی می‌دهیم.



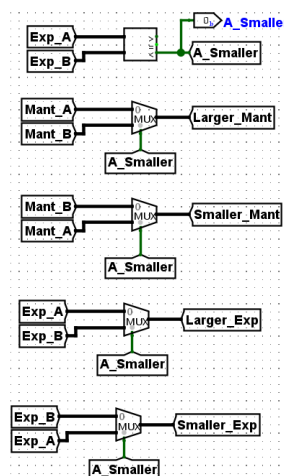
شکل ۳: مدار بخش Unpacker

۲.۳ Exp_Aligner

ابتدا باید نما دو عملوند با یکدیگر برابر شوند. این روند در زیر مدار Exp_Aligner در سه فاز اصلی انجام میشود:

فاز اول: تشخیص عملوند کوچکتر و تفکیک سیگنال‌ها

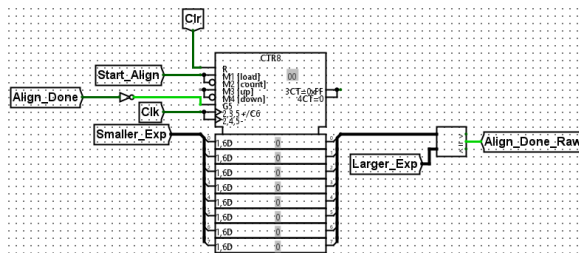
در اولین قدم، مدار باید تشخیص دهد که کدام عملوند نیاز به شیفت دارد. مطابق شکل ۴، با استفاده از یک Comparator مقادیر Exp_A و Exp_B را با یکدیگر مقایسه کرده و سیگنال کنترلی A_Smaller را تولید می‌کنیم. سپس به کمک مجموعه‌ای از Multiplexerها، مقادیر بزرگتر و کوچکتر را از یکدیگر تفکیک کرده و آن‌ها را تحت عناوین Larger_Exp، Smaller_Exp، Larger_Mant و Smaller_Mant به مراحل بعدی هدایت می‌کنیم.



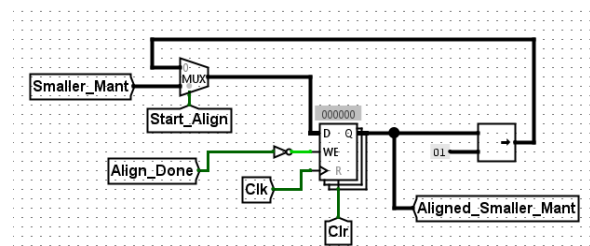
شکل ۴: مدار مقایسه‌گر و تفکیک‌کننده عملوندها در بلوک Exp_Aligner

فاز دوم: حلقه شیفت

بخش اصلی هم‌ترازی در این مرحله قرار دارد. ما باید Mantissa مربوط به عدد کوچکتر را تا جایی به سمت راست شیفت دهیم که نمای آن با نمای عدد بزرگتر برابر شود. ما از یک Register برای نگهداری مانتیس و از یک Counter برای نگهداری نمای عدد کوچکتر استفاده کردیم. با فعال شدن سیگنال Start_Align، مقادیر اولیه در آنها ثبت می‌شوند. پس از آن، با هر Clk، مانتیس یک بیت به راست شیفت خورده و شمارنده یک واحد به نما اضافه می‌کند. این حلقه تا زمانی که سیگنال Align_Done_raw صفر باشد، تکرار می‌شود.



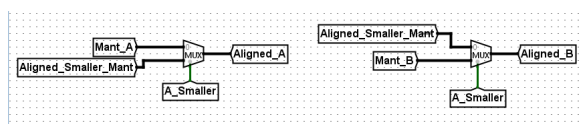
شکل ۶: شمارنده برای هم‌ترازی نما



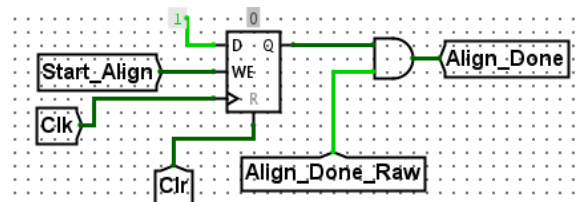
شکل ۵: حلقه شیفت به راست برای مانتیس کوچکتر

فاز سوم: تولید سیگنال پایان و مسیریابی خروجی

برای توقف حلقه، یک Comparator به طور پیوسته مقدار Counter را با Larger_Exp مقایسه می‌کند. به محض برابری، سیگنال Align_Done_Raw یک می‌شود. برای جلوگیری از اشتباه ناشی از ۰ بودن مقادیر در ابتدای کار، ما این سیگنال را وارد یک D Flip-Flop کردیم تا سیگنال نهایی Align_Done همراه با کلاک تولید شود. در نهایت، با استفاده از Multiplexerهای خروجی، مانتیس شیفت‌خورده (Aligned_Smaller_Mant) را بسته به وضعیت بزرگی یا کوچکی هر یک از ورودی‌ها با کمک سیگنال A_Smaller یا A_Bigger به جایگاه اصلی خود یعنی Aligned_A یا Aligned_B هدایت کرده و عملوند بزرگتر را نیز دست‌نخورده به خروجی منتقل می‌کنیم.



شکل ۸: مسیریابی نهایی برای خروجی‌های Aligned



شکل ۷: مدار سیگنال پایان

۳.۳ Big_ALU

پس از هم‌ترازی نماها، باید محاسبات را روی بخش کسری انجام دهیم. بلوک Big_ALU وظیفه دارد بر اساس سیگنال کنترل Op، مانیتیس‌ها را جمع یا تفریق کند. روند کار این بلوک در سه مرحله خلاصه می‌شود:

مرحله اول: تشخیص مانیتیس بزرگتر و تفکیک سیگنال‌ها

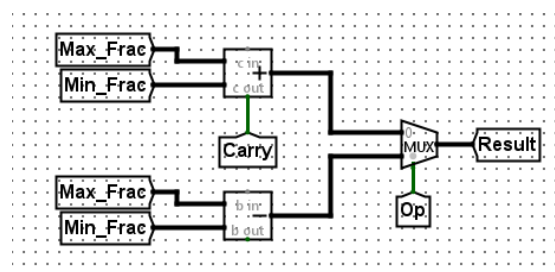
برای جلوگیری از تولید اعداد منفی موقت در عملیات تفریق، منطق مدار را به گونه‌ای طراحی کردیم که همواره عدد کوچکتر از عدد بزرگتر کم شود. همان‌طور که در شکل ۹ مشخص است، ابتدا با استفاده از یک Comparator مقادیر Frac_A و Frac_B مقایسه شده و سیگنال A_Smaller تولید می‌شود. در ادامه، دو Multiplexer این مقادیر را به دو مسیر Max_Frac و Min_Frac تفکیک می‌کند.

مرحله دوم: محاسبه همزمان و انتخاب خروجی

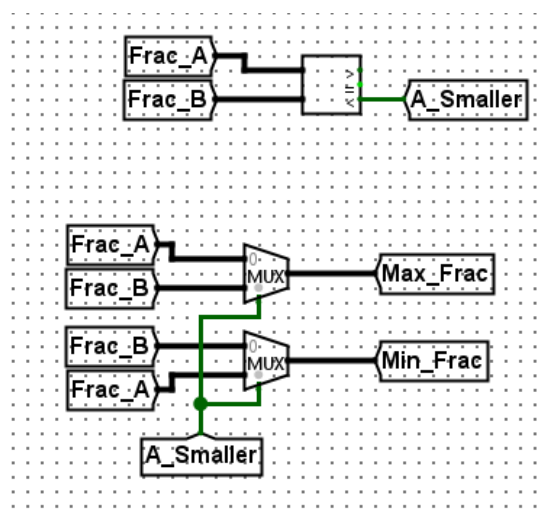
مقادیر تفکیک شده مطابق شکل ۱۰ به صورت همزمان وارد یک Adder و یک Subtractor می‌شوند. قطعه جمع‌گر علاوه بر حاصل عملیات، در صورت نیاز یک بیت Carry نیز تولید می‌کند. در آخرین نقطه از این بخش، یک Multiplexer با بیت سلکت از Op، خروجی نهایی را بین نتیجه جمع و تفریق انتخاب کرده و آن را روی Result قرار می‌دهد.

مرحله سوم: منطق فیلتر سیگنال Carry

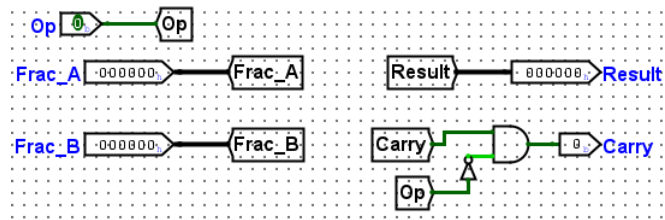
بیت کری در عملیات تفریق ممیز شناور کاربردی برای ما ندارد و تنها در حالت جمع است که تولید Carry منجر به نیاز به شیفت در بلوک نرمال‌ساز می‌شود. به همین دلیل، طبق منطق نشان داده شده در شکل ۱۱، با استفاده از ترکیب گیت‌های AND و NOT شرطی قرار دادیم که سیگنال Carry تنها در صورتی که عملیات از نوع جمع باشد می‌تواند به خروجی برود.



شکل ۱۰: محاسبه جمع و تفریق و انتخاب براساس اوپریشن



شکل ۹: مدار مقایسه و تفکیک مانیتیس‌ها



شکل ۱۱: ورودی/خروجی و منطق فیلتر Carry

۴.۳ Normalizer

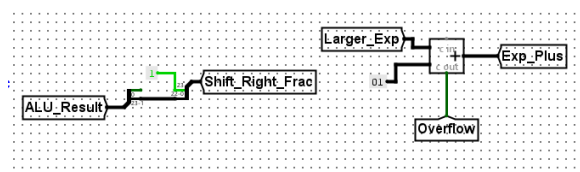
خروجی تولید شده توسط Big_ALU ممکن است در فرمت نرمالایز نباشد. واحد Normalizer وظیفه دارد با شیفت دادن Mantissa و اصلاح Exponent، عدد را به فرمت استاندارد IEEE-754 بازگرداند. عملکرد این زیرمدار در سه فاز زیر انجام می‌شود:

فاز اول: پیش‌پردازش، مدیریت Carry و تشخیص Overflow

اگر عملیات از نوع جمع باشد، ممکن است خروجی ALU دارای بیت کری باشد. در این حالت، مانیتیس باید یک بیت به راست شیفت خورده و نما یک واحد افزایش یابد. ما از اسپلیتر استفاده کردیم که ALU_Result را به راست شیفت داده (Shift_Right_Frac) و یک مقدار Larger_Exp را یک واحد افزایش می‌دهد (Exp_Plus). پایه Carry Out در این جمع کننده، مستقیماً به عنوان سیگنال خطای Overflow به خروجی سیستم متصل شده است. سپس مشخص است، با استفاده از دو Multiplexer و بر اساس وضعیت پین Carry، تصمیم می‌گیریم که مقادیر اولیه برای بارگذاری در حلقه نرمال‌سازی (Loading_Exp و Loading_Mant) مقادیر اصلاح شده باشند یا همان مقادیر خام ALU.



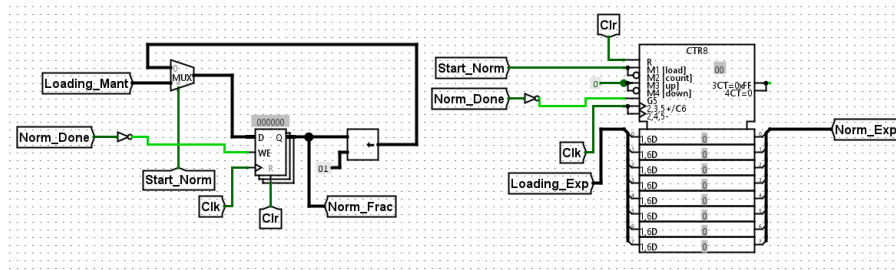
شکل ۱۳: مالتی پلکسر ها برای حلقه نرمال‌سازی



شکل ۱۲: مدار اصلاح Carry و تولید خطای Overflow

فاز دوم: حلقه ترتیبی شیفت به چپ

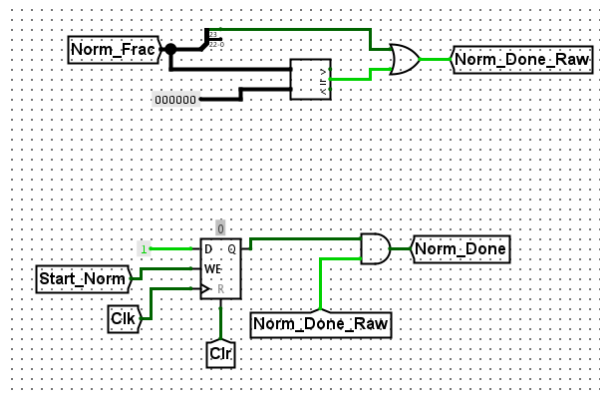
در صورتی که عملیات تفریق باشد، ممکن است چندین صفر در سمت چپ مانیتیس ایجاد شود. برای رفع این مشکل، ما مقادیر اولیه‌ای که در فاز قبل تفکیک شدند را وارد یک حلقه ترتیبی می‌کنیم. مقدار مانیتیس در یک Register و مقدار نما در یک Counter هشت بیتی ثبت می‌شود. با دریافت سیگنال Start_Norm، مدار با هر لبه کلاک، مانیتیس را یک بیت به چپ شیفت داده و شمارنده نما را یک واحد کاهش می‌دهد. این فرآیند ترتیبی تا زمانی که با ارزش‌ترین بیت در مانیتیس برابر با ۱ شود، ادامه می‌یابد.



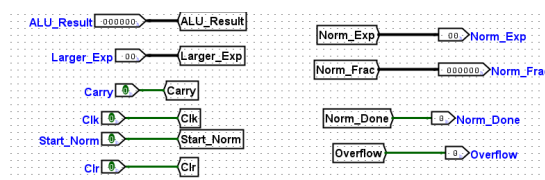
شکل ۱۴: حلقه ترتیبی شامل رجیستر شیفت به چپ و شمارنده کاهشی نما

فاز سوم: منطق تشخیص پایان و همگام‌سازی

حلقه نرمال‌سازی باید دقیقاً در زمان مناسب متوقف شود. مطابق منطق پیاده‌سازی شده در شکل ۱۵، شرط توقف ما دو حالت دارد: یا با ارزش‌ترین بیت مانتیس یک شود (عدد نرمال شده باشد)، و یا کل مانتیس برابر با صفر باشد (برای جلوگیری از شیفت‌های بی‌نهایت در زمانی که حاصل محاسبه دقیقاً صفر است). این دو شرط از طریق یک گیت OR سیگنال Norm_Done_Raw را می‌سازند. در نهایت، برای جلوگیری از فعال شدن سیگنال Norm_Done در ابتدای کار سیستم (که هنوز مقایسه در دست است)، از یک فلیپ فلاپ با ورودی ۱ استفاده می‌کنیم که سیگنال enable آن به سیگنال شروع سیستم وصل است؛ خروجی این فلیپ فلاپ با سیگنال Norm_Done_Raw از یک گیت And عبور میکند و سیگنال نهایی Norm_Done را تولید میکند که در نهایت به واحد کنترل ارسال می‌شود.



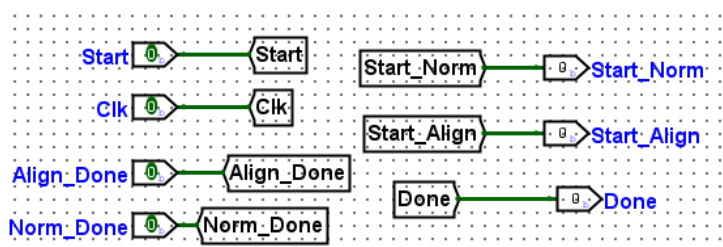
شکل ۱۵: منطق توقف حلقه نرمال‌سازی و سیگنال Done



شکل ۱۶: پین های ورودی و خروجی

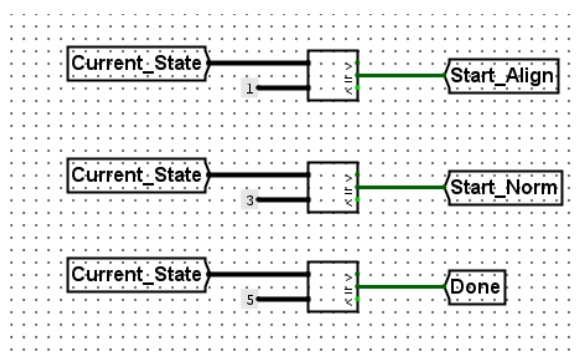
۵.۳ واحد CU

برای هماهنگ‌سازی تمام این بخش‌ها، یک State Machine طراحی کردیم. این واحد با دریافت سیگنال Start، ابتدا به Exp_Aligner فرمان شروع می‌دهد. پس از دریافت Align_Done، مدار وارد فاز محاسبه شده و سپس فرمان Start_Norm را برای بلوک نرمال‌ساز صادر می‌کند. در نهایت با دریافت تأییدیه اتمام نرمال‌سازی، سیگنال Done فعال می‌شود تا آماده بودن خروجی نهایی را به سیستم گزارش دهد.

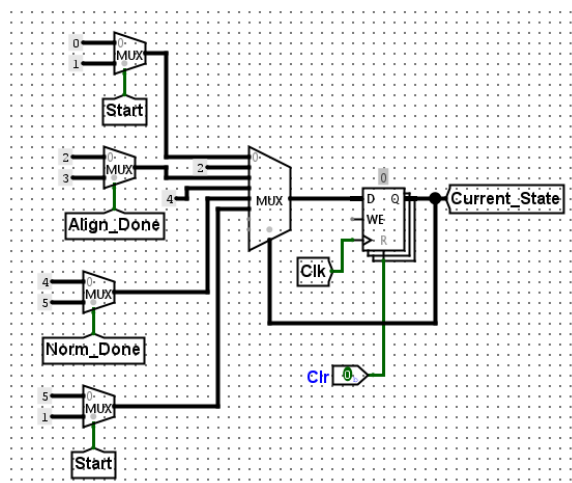


شکل ۱۷: سیگنال‌های ورودی و خروجی

در ماشین حالت، ۶ حالت داریم، منطق اصلی واحد کنترل با استفاده از تعدادی مالتی پلکسر و یک فلیپ فلاپ سه بیتی پیاده‌سازی شده است؛ به طوری که در هر مرحله بر اساس حالت فعلی و سیگنال‌های ورودی تصمیم گرفته می‌شود که حالت بعدی چه حالتی باشد.



شکل ۱۹: فعال‌سازی سیگنال‌های خروجی براساس حالت فعلی و با استفاده از مقایسه‌کننده‌ها



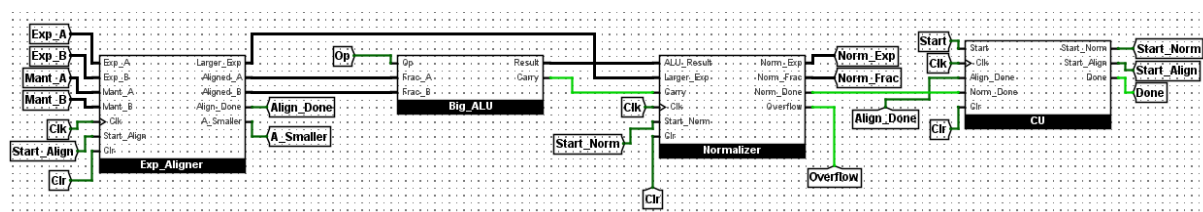
شکل ۱۸: منطق اصلی واحد کنترل

۶.۳ Main

عملکرد این مدار را می‌توان در سه بخش زیر خلاصه کرد:

بخش اول: یکپارچه‌سازی DP و CU

مطابق شکل ۲۰، زیرمدارهای Exp_Aligner، Big_ALU، Normalizer و Normalizer به صورت زنجیروار به یکدیگر متصل شده‌اند تا مسیر داده به درستی طی شود. در کنار آن‌ها، بلوک CU قرار دارد که با ارسال سیگنال‌های Start_Align و Start_Norm و دریافت تاییدیه‌های Align_Done و Norm_Done، اجرا را هماهنگ کرده و در نهایت سیگنال Done را ارسال می‌کند.



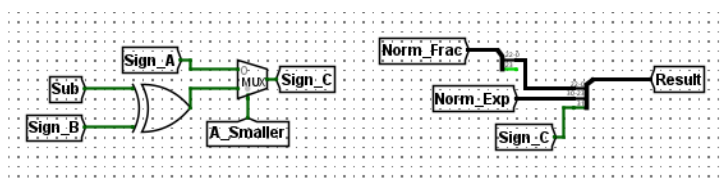
شکل ۲۰: اتصال زیرمدارها و یکپارچه‌سازی DP با CU

بخش دوم: منطق تولید سیگنال Op و تعیین علامت نهایی

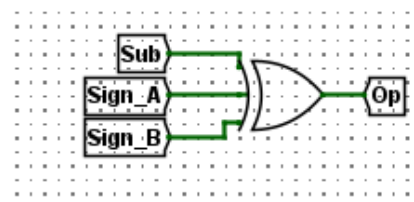
مدار ALU برای تشخیص اینکه باید عملیات جمع انجام دهد یا تفریق، به سیگنال Op نیاز دارد. ما این سیگنال را با استفاده از یک گیت XOR سه‌ورودی که پایه‌های Sub، Sign_A و Sign_B به آن متصل‌اند، تولید کردیم. همچنین برای تعیین علامت نهایی نتیجه (Sign_C)، منطق به این صورت است که علامت جواب باید مشابه علامت عملوند بزرگتر باشد. بر همین اساس، از یک Multiplexer استفاده کردیم که توسط سیگنال A_Smaller کنترل می‌شود؛ اگر A بزرگتر باشد، Sign_A عبور می‌کند و اگر B بزرگتر باشد، حاصل XOR بین Sign_B و پین Sub به عنوان علامت نهایی انتخاب می‌گردد.

بخش سوم: تولید خروجی ۳۲ بیتی

در آخرین گام، پس از اینکه Normalizer کار خود را به اتمام رساند، باید مقادیر تفکیک‌شده را مجدداً به فرمت استاندارد IEEE-754 بازگردانیم. برای این کار، از یک Splitter در حالت معکوس استفاده کردیم. ۲۳ بیت حاصل از Norm_Frac، ۸ بیت Norm_Exp و تک‌بیت Sign_C در کنار هم قرار گرفته و خروجی یکپارچه‌ی ۳۲ بیتی Result را می‌سازند تا بر روی پین C قرار گیرد.



شکل ۲۲: منطق تعیین علامت نهایی و خروجی ۳۲ بیتی



شکل ۲۱: مدار تولید سیگنال Op

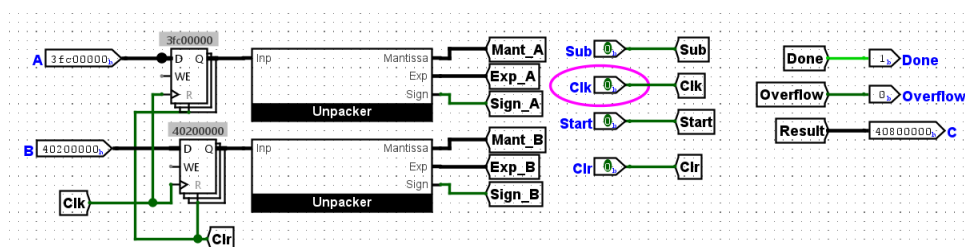
۴ نکات و جزئیات

- سیگنال کلیر در دستور کار وجود نداشت اما برای راحت تر شدن انجام تست های متوالی ، این سیگنال را به مدار اضافه کردیم.
- در صورت سوال بجای سیگنال Done ، سیگنال End وجود داشت اما به دلیل محدودیت نام گذاری در لاجیسیم ، این تفاوت ایجاد شد
- مدار براساس اختلاف نما ها و ورودی های داده شده ، با تعداد کلاک های متفاوت خروجی نهایی را محاسبه میکند

۵ بررسی Test Case ها

۱.۵ تست اول: جمع ساده

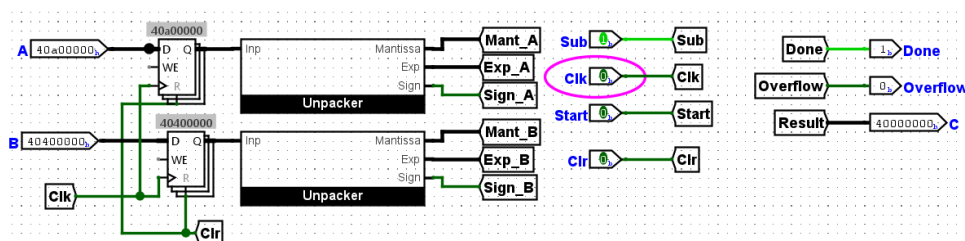
Sub=0 ، B(2.5)=40200000 ، A(1.5)=3FC00000
overflow=0 ، C(4.0)=40800000



شکل ۲۳: تست اول

۲.۵ تست دوم: تفریق ساده و نرمال سازی

Sub=1 ، B(3.0)=40400000 ، A(5.0)=40A00000
overflow=0 ، C(2.0)=40000000

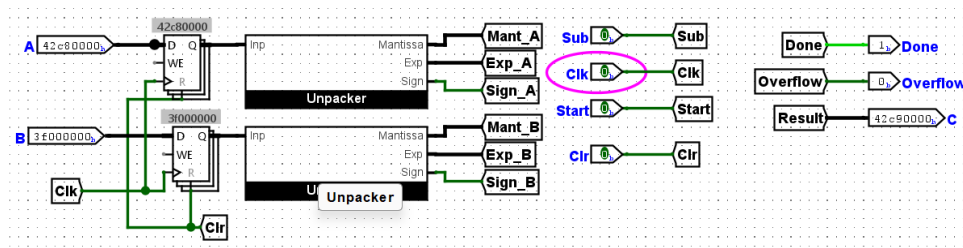


شکل ۲۴: تست دوم

۳.۵ تست سوم: اختلاف نمای زیاد

Sub=0 ، B(0.5)=3F000000 ، A(100.0)=42C80000

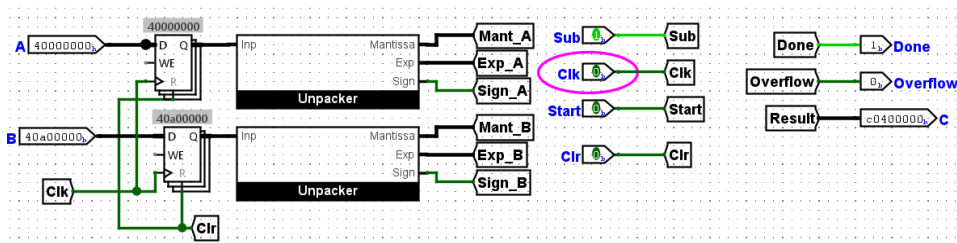
overflow=0 , C(100.5)=42C90000



شکل ۲۵: تست سوم

۴.۵ تست چهارم: تولید عدد منفی

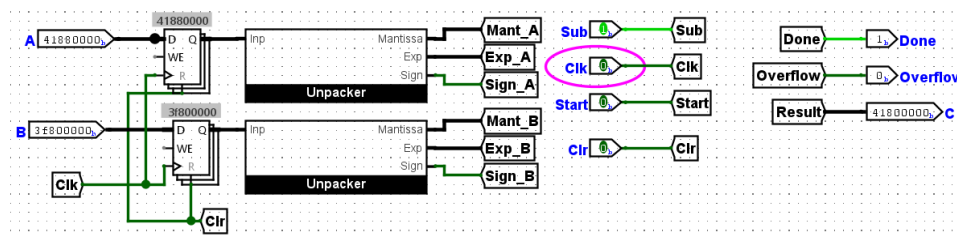
Sub=1 , B(5.0)=40A00000 , A(2.0)=40000000
overflow=0 , C(-3.0)=C0400000



شکل ۲۶: تست چهارم

۵.۵ تست پنجم: نرمال سازی عمیق

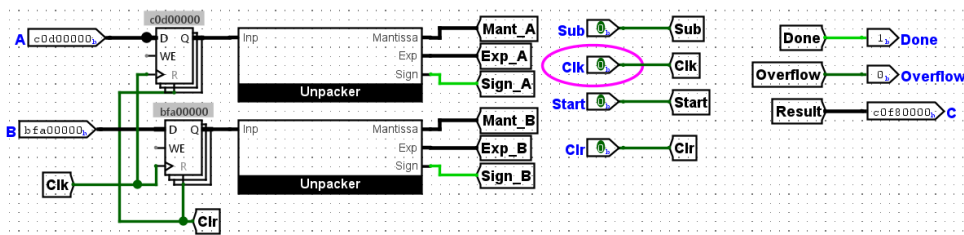
Sub=1 , B(1.0)=3F80000 , A(17.0)=41880000
overflow=0 , C(16.0)=41800000



شکل ۲۷: تست پنجم

۶.۵ تست ششم: جمع دو عدد منفی

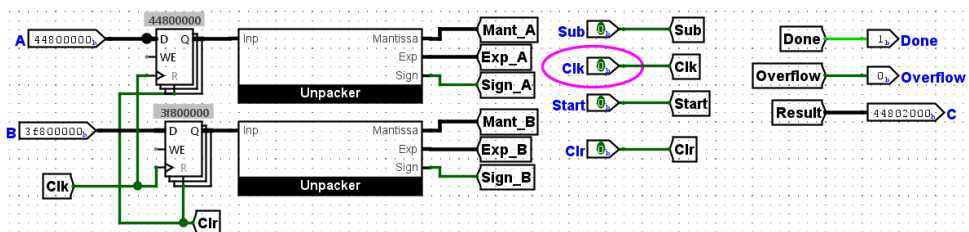
Sub=0 , B(-1.25)=BFA0000 , A(-6.5)=C0D00000
overflow=0 , C(-7.75)=C0F80000



شکل ۲۸: تست ششم

۷.۵ تست هفتم: اختلاف نمای بسیار زیاد

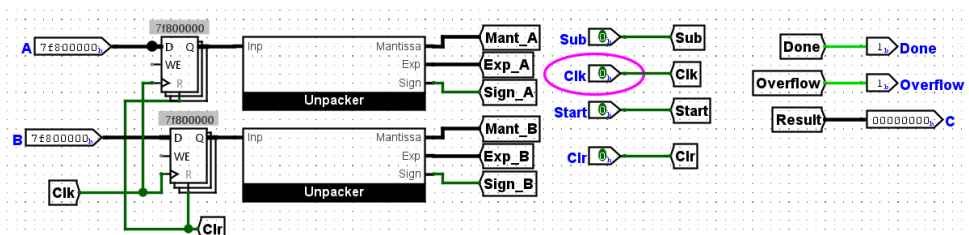
Sub=0 , B(1)=3F80000 , A(1024.0)=44800000
overflow=0 , C(1025.0)=44802000



شکل ۲۹: تست هفتم

۸.۵ تست هشتم: اورفلو

Sub=0 , B=7F80000 , A=7F800000
overflow=1 , C=00000000



شکل ۳۰: تست هشتم

۶ نتیجه گیری

در این آزمایش، ما توانستیم مدار جمع و تفریق کننده ممیز شناور ۳۲ بیتی را بر اساس استاندارد IEEE-754 به صورت ترتیبی طراحی و پیاده سازی کنیم. با تقسیم کردن مدار به بلوک های مختلف مثل Unpacker، Normalizer و Big_ALU، Exp_Aligner، توانستیم کارها را به بخش های کوچک تر و قابل مدیریت تقسیم کنیم. مدار ما با روشن شدن سیگنال Done اتمام کار را اعلام میکند و در صورت بروز اورفلو نیز با روشن کردن سیگنال Overflow این خطا را اعلام میکند.